

DYNAMIC WEB TECHNOLOGIES



RouteMemo Fitness Tracker

React Native | Expo

April 2026

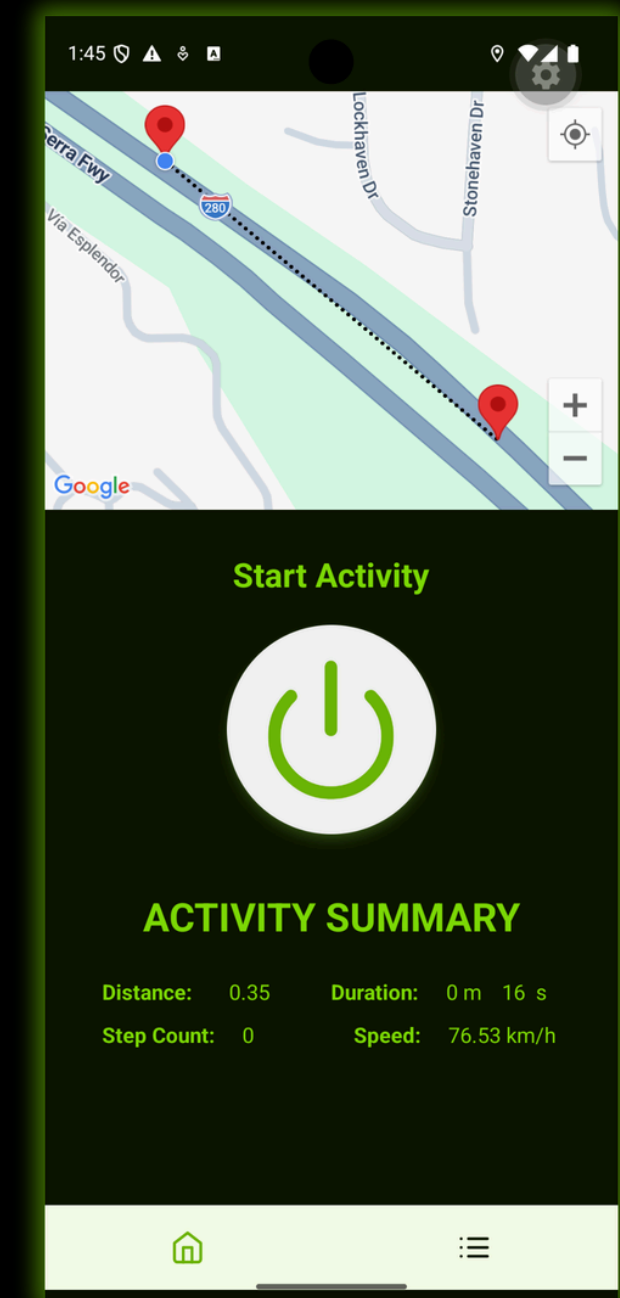
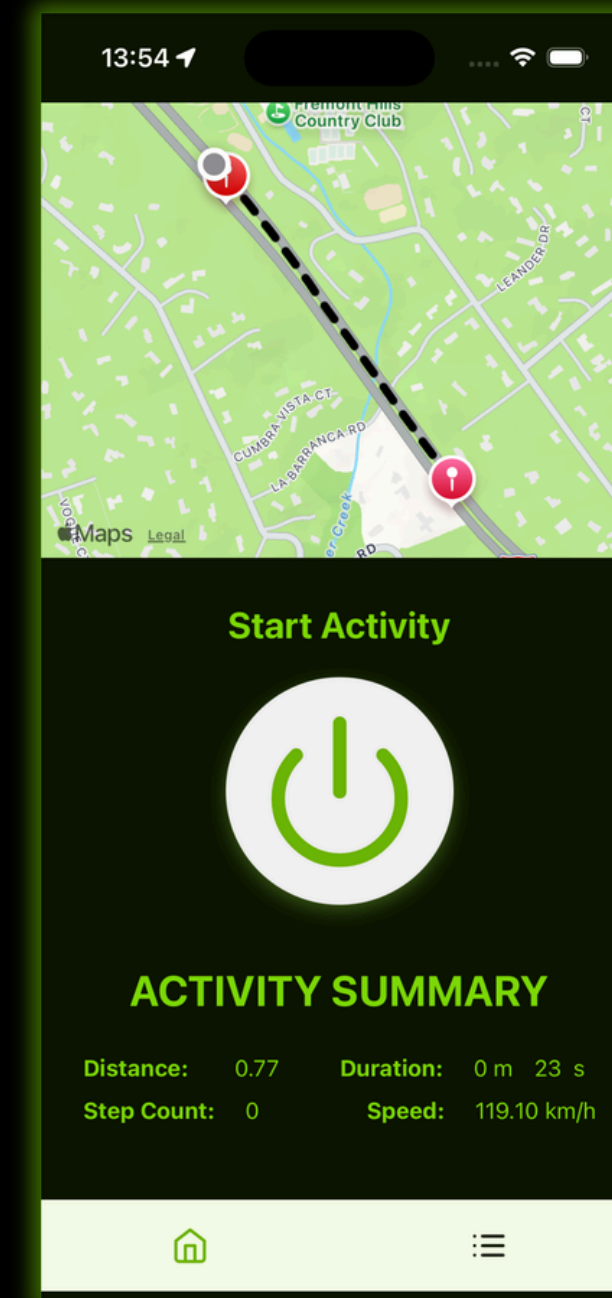
● Andrew Sullivan



● BSC Web & Mobile Development

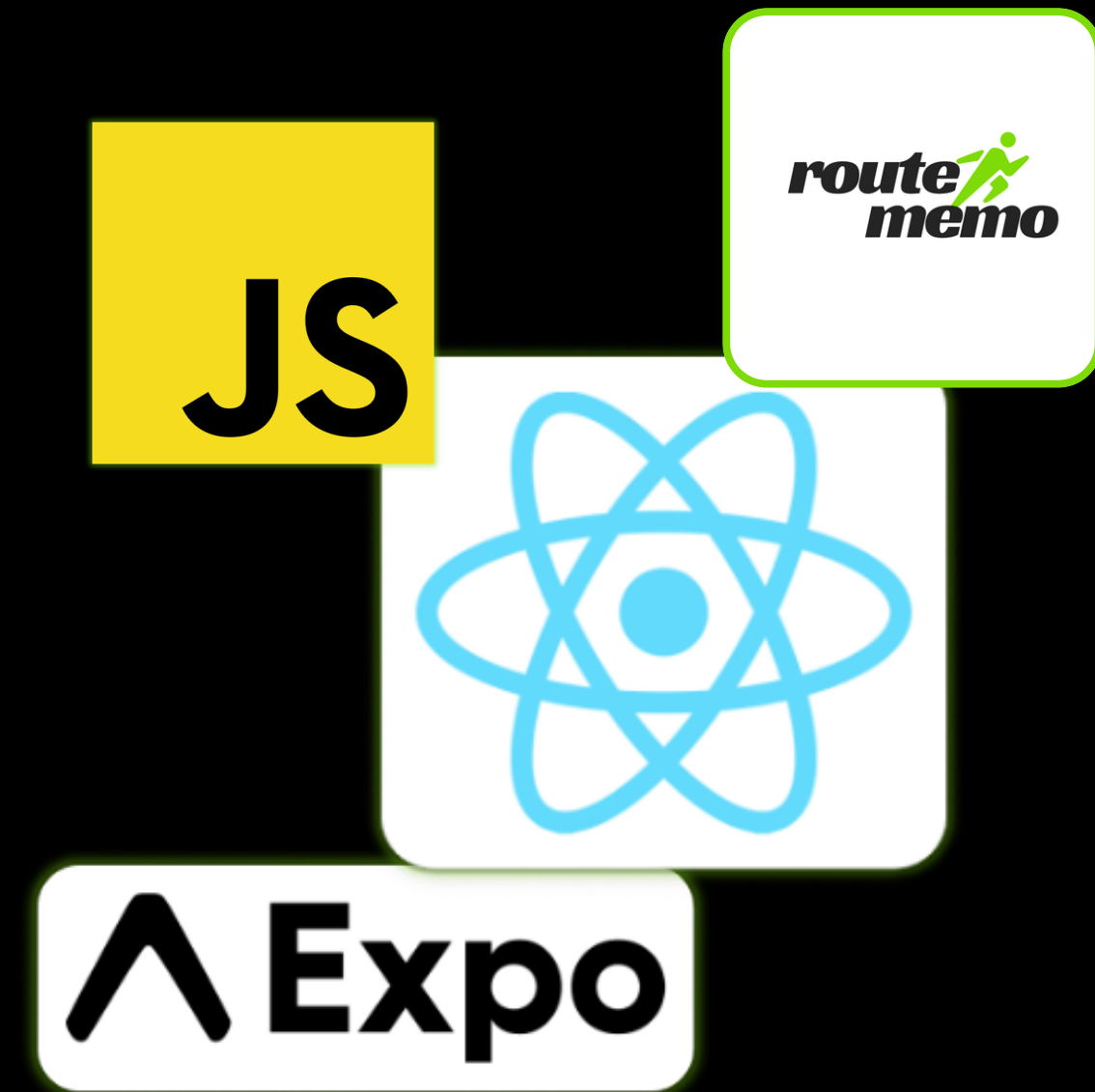
Project Introduction

- RouteMemo is a cross-platform mobile application built with React Native / Expo frameworks
- The app can be used to track exercise, and show additional information such as speed, duration and distance travelled, steps using the devices' pedometer, as well as mapping the route coordinates to a native map
- Previously saved routes can be accessed and notes and photos can be added
- The application works across both iOS and Android devices accessing their native device features
- The main features of the app work offline with users still able to record exercise and view and edit past activities



Architecture

- RouteMemo was developed using the Expo Framework which is a framework built on React Native and uses JavaScript as the underlying language
- React Native allows a single code-base to be used across both iOS and Android reducing development time and simplifying mobile app production
- The application is component based to separate UI concerns with state management and data handling
- State is managed using React hooks and data is persisted on devices using AsyncStorage
- Native APIs of location, sensors for pedometer and camera are accessed using expo-location, expo-sensors and expo-camera

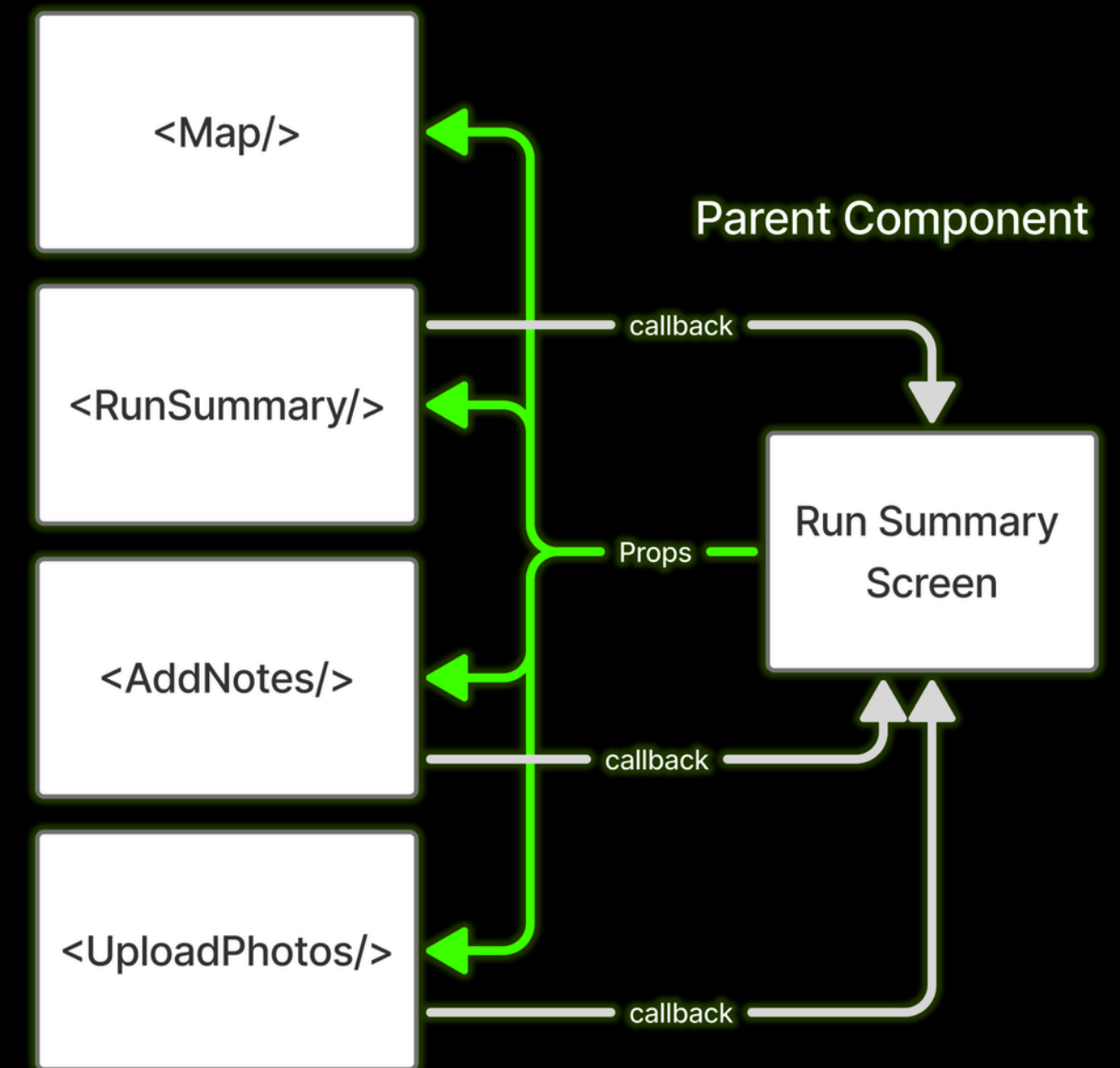


Component Based Structure

- RouteMemo follows a component based structure where UI logic is separated from data logic supporting code easy code maintenance and re-usability
- In the example below the RunDetailsScreen is the parent (container) component controlling the state and handling AsyncStorage logic and passing data and functions to its children
- In most cases (apart from <Map/> here) data is sent back to the parent through callback functions to trigger an action

```
<ScrollView contentContainerStyle={{ flexGrow: 1, paddingBottom: 60 }}>
  <Map route={run.route} />
  <RunSummary
    run={run}
    onOpenCamera={handleOpenCamera}
  />
  <AddNotes run={run} onSaveNote={saveNote} />
  <UploadPhoto
    photos={run.photos}
    onOpenCamera={handleOpenCamera}
    onDeletePhoto={deletePhoto}
  />
  <DeleteButton id={run.id} onDelete={deleteRun} />
</ScrollView>
```

Child Components



Navigation

- RouteMemo uses a combination of tab and stack navigation to guide users through the app
- The bottom tab navigator provides access to the two main screens of “Home” where users start and stop activities, and “Runs” which lists all the users previous activities
- Icons are rendered dynamically based on the route name and change focus colour dependent on whether the screen is active or not
- Within the Runs screen a stack navigator is used to move from the list of runs to individual run details
- When the run is tapped then the runId is passed to the RunDetails screen using the useNavigation hook
- Then the exercise details are retrieved based on that Id from AsyncStorage and displayed on screen load once the screen has rendered using the useEffect hook

```
<RunListTable
  activities={activities}
  onPressItem={({run}) =>
    navigation.navigate("RunDetails", { runId: run.id })
  }
/>
```

RunList -->
RunDetails

```
useEffect(() => {
  const loadRun = async () => {
    const existing = await AsyncStorage.getItem("activities");
    const parsed = existing ? JSON.parse(existing) : [];
    const selectedRun = parsed.find((item) => item.id === runId);

    setRun(selectedRun);
  };

  loadRun();
}, [runId]);
```


CRUD Operations & Persistent Storage

- RouteMemo uses local AsyncStorage which is a key-value storage type to persist data and has CREATE, READ, UPDATE and DELETE functionality for managing exercise data
- When users start a new exercise a new instance is CREATED and stored to AsyncStorage
- The activityLog object shown below defines the activity dataset to be stored before being parsed and stringified, then pushed to AsyncStorage
- The route data is recorded every 5 metres or every 3 seconds (whichever comes first) so that that data can be rendered to the map as a route line, and looks like the bottom image when stored

```
const activityLog = {
  id: Date.now().toString(),
  route,
  startPoint: route[0],
  endPoint: route[route.length - 1],
  duration,
  distance,
  speed,
  steps,
  timestamp: startTime.toISOString(),
  notes: "Add your notes here",
};
```

```
try {
  // get existing activities from storage or start with empty array
  const existing = await AsyncStorage.getItem("activities");
  const parsed = existing ? JSON.parse(existing) : [];

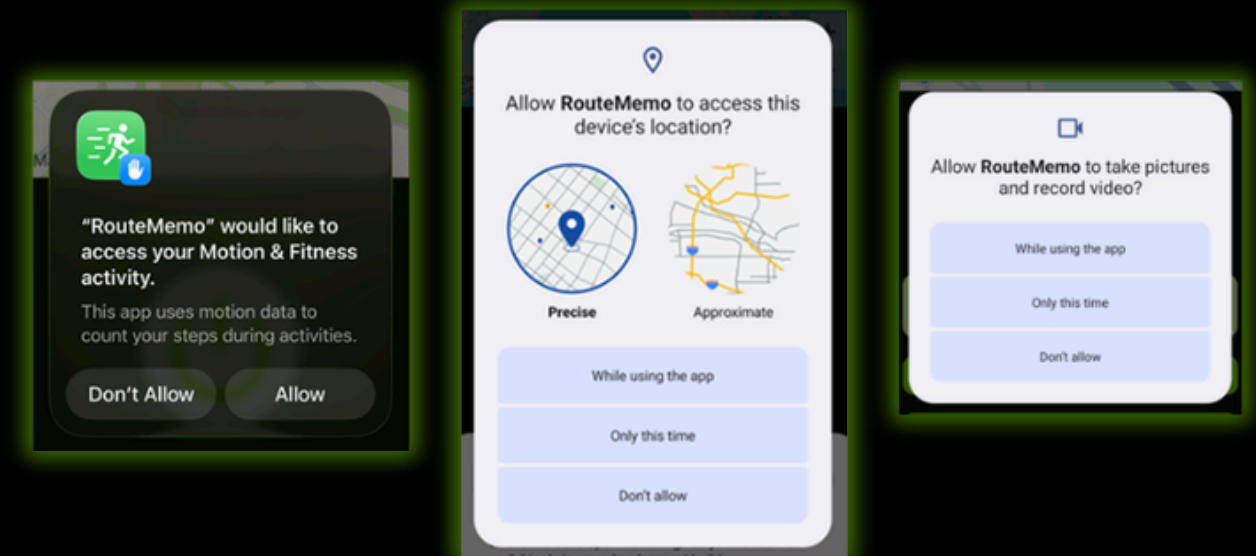
  // push new activity to array and save back to storage
  parsed.push(activityLog);

  // stringify and save to storage
  await AsyncStorage.setItem("activities", JSON.stringify(parsed));
}
```

```
LOG Activity saved: {"distance": 453.4422689876394, "duration": 13.353, "endPoint": {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 341.37, "latitude": 37.52889894, "longitude": -122.35986941, "speed": 34.66}, "id": "1776106957774", "notes": "Add your notes here", "route": [{"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.14, "latitude": 37.52527918, "longitude": -122.35759682, "speed": 33.98}, {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.49, "latitude": 37.52552611, "longitude": -122.35781861, "speed": 34.13}, {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.49, "latitude": 37.52578192, "longitude": -122.35804173, "speed": 34.33}, {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.49, "latitude": 37.52603468, "longitude": -122.35827114, "speed": 34.33}], "startPoint": {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.14, "latitude": 37.52527918, "longitude": -122.35759682, "speed": 33.98}, "timestamp": "2023-09-14T14:30:13.353Z", "steps": 1, "speed": 34.66, "startPoint": {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.14, "latitude": 37.52527918, "longitude": -122.35759682, "speed": 33.98}, "endPoint": {"accuracy": 5, "altitude": 0, "altitudeAccuracy": -1, "heading": 324.49, "latitude": 37.52578192, "longitude": -122.35804173, "speed": 34.13}, "duration": 13.353, "distance": 453.4422689876394, "speed": 34.66, "steps": 1, "timestamp": "2023-09-14T14:30:13.353Z", "notes": "Add your notes here", "id": "1776106957774"}
```

Native APIs & App Permissions

- RouteMemo uses three native APIs to enhance its functionality
 - Geolocation using Expo Location
 - Camera using Expo Camera
 - Pedometer using Expo Sensors (only works on physical devices)
- To access the devices native APIs user permissions need to be granted
- In the app.config.js file we need to explicitly tell the device that it wants to access these, and in the UI check if these are granted before using



```
ios: {
  bundleIdentifier: "com.thewebdeveloperghlasgow.RouteMemo2",
  infoPlist: {
    NSLocationWhenInUseUsageDescription:
      "This app uses your location to track your route and provide you with a summary of your activity.",
    NSMotionUsageDescription:
      "This app uses motion data to count your steps during activities.",
    "ITSAppUsesNonExemptEncryption": false,
  },
},
android: {
  package: 'com.thewebdeveloperghlasgow.RouteMemo2',
  permissions: ['ACCESS_FINE_LOCATION', 'ACTIVITY_RECOGNITION'],
  adaptiveIcon: {

```

```
"expo-camera",
{
  cameraPermission: "Allow $(PRODUCT_NAME) to access your camera",
  microphonePermission: "Allow $(PRODUCT_NAME) to access your microphone",
  recordAudioAndroid: true,
  barcodeScannerEnabled: true
}
```

```
// request location and pedometer permissions on mount
useEffect(() => {
  (async () => {
    const { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== "granted") {
      alert("Permission to access location is required to use the app");
    }

    // pedometer
    const { status: pedometerStatus } =
      await Pedometer.requestPermissionsAsync();

    if (pedometerStatus !== "granted") {
      alert("Permission to access pedometer is required to use the app");
    }
  })();
}, []);
```

CRUD Operations & Persistent Storage

- Data from AsyncStorage is READ in multiple places. The RunListTable maps selected activity data to a FlatList so the user can pick a run and view it in more detail
- Users can DELETE unwanted runs from AsyncStorage from the RunDetailsScreen
 - When the DeleteButton is triggered the component receives the id of the activity as a prop from the parent component
 - This triggers the handleDelete function in the child component
 - On press of confirm delete the deleteRun function is triggered in the parent component removing the item from AsyncStorage

```
<DeleteButton id={run.id} onDelete={deleteRun} />
```

```
function DeleteButton({ id, onDelete }) {  
  const handleDelete = () => {  
    Alert.alert(  
      "Delete Exercise",  
      "Are you sure, this cannot be undone?",  
      [  
        { text: "Cancel", style: "cancel" },  
        {  
          text: "Delete",  
          style: "destructive",  
          onPress: () => onDelete(id),  
        }  
      ],  
    );  
  };  
}
```

```
const deleteRun = async (id) => {  
  const existing = await AsyncStorage.getItem("activities");  
  const parsed = existing ? JSON.parse(existing) : [];  
  
  const updated = parsed.filter((item) => item.id !== id);  
  
  await AsyncStorage.setItem("activities", JSON.stringify(updated));  
  
  Alert.alert("Deleted", "The run was deleted");  
  navigation.goBack();  
};
```


Dynamic Maps

- Maps are a key feature of RouteMemo, displaying the users position when they are recording an activity using the react-native-maps third party plugin library to render to the UI
- The Map component is a re-usable component that is used in both the Start Screen and the Summary Screens
- The coordinate data is stored at intervals so it can be used to re-render the Polyline route on individual runs
- The useState hook is used to store the user's current location, allowing the map to update dynamically.
- useRef is used to create a reference to the MapView component
- And useEffect is used to respond to changes in that route data, automatically updating and adjusting the map view to the new coordinates as recorded using the fitToCoordinates method provided by the library
- Conditional rendering is applied to ensure that in different map views either the route being created is displayed or the route that exists is rendered

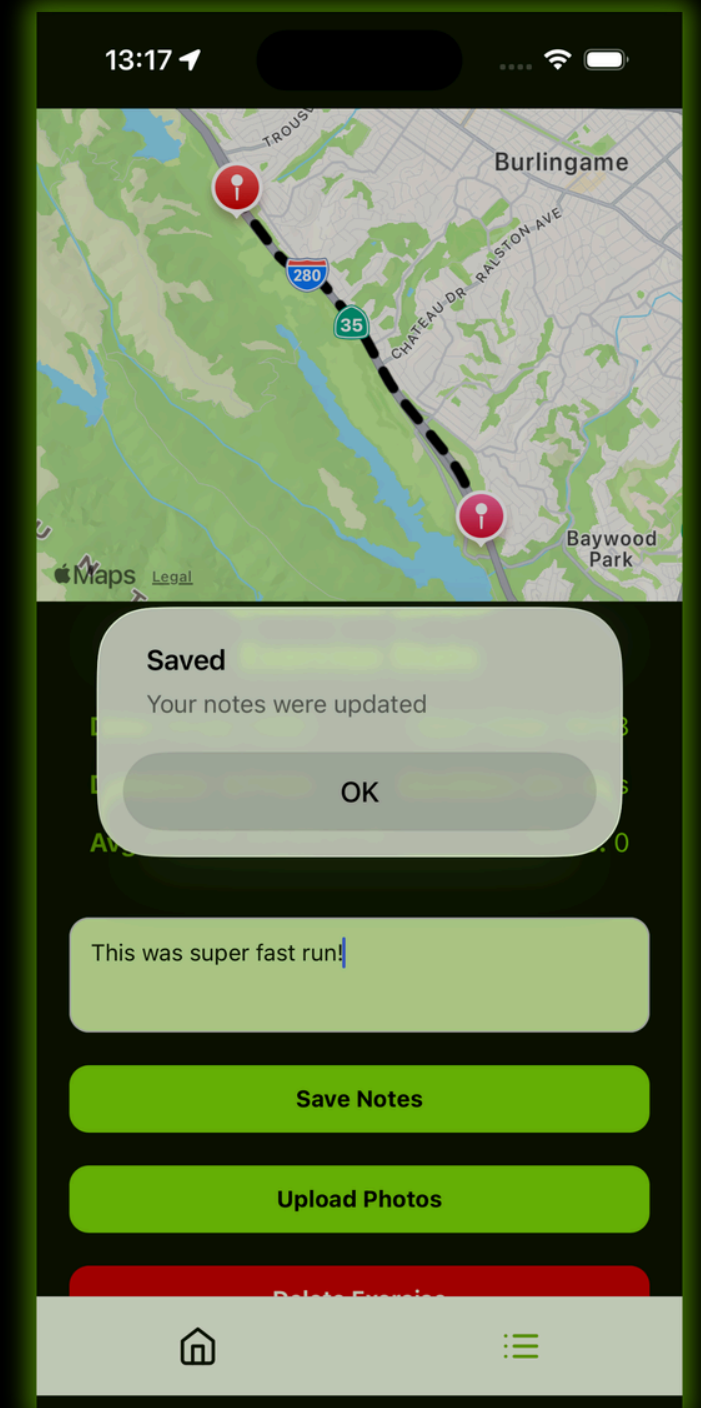
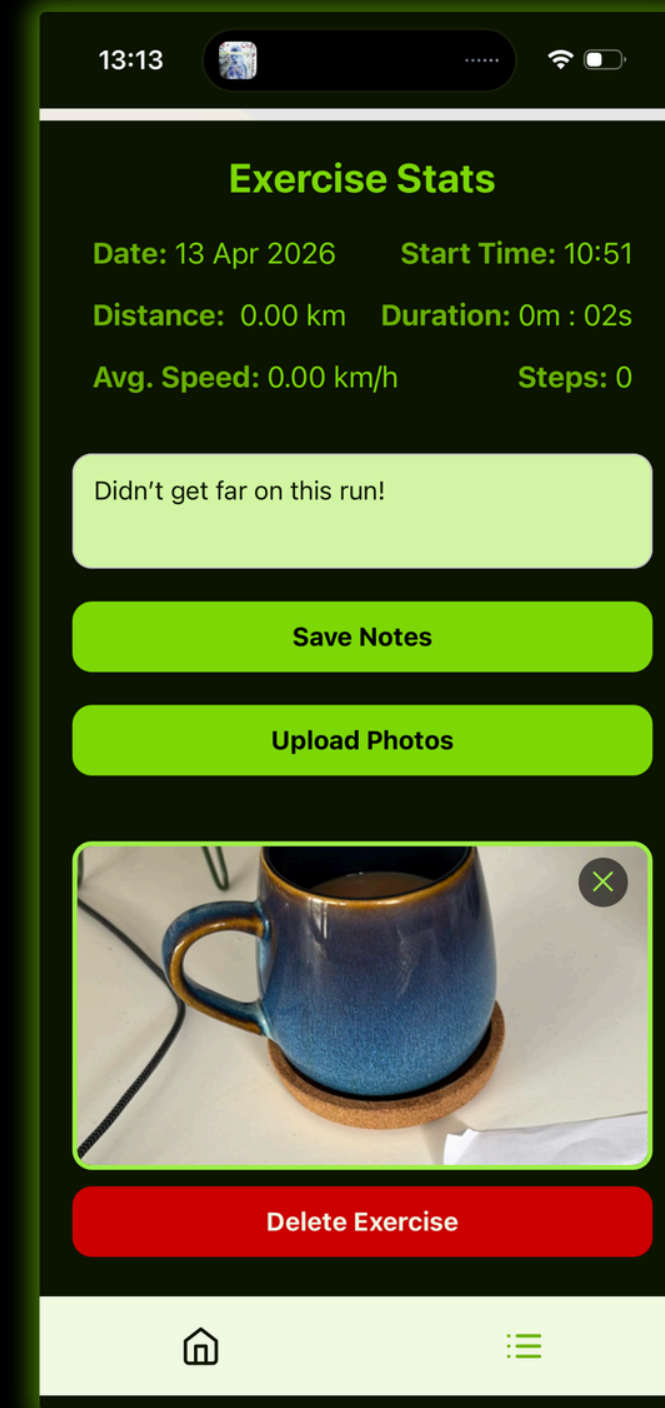


```
const mapRef = useRef(null);  
const [location, setLocation] = useState(null);
```

```
useEffect(() => {  
  if (route && route.length > 0) {  
    mapRef.current?.fitToCoordinates(route, {  
      edgePadding: { top: 50, right: 50, bottom: 50, left: 50 },  
      animated: true,  
    });  
  }  
}, [route]);
```

Adding Notes & Photos

- Users can add notes and photos to their activities after they are saved
- If notes or photos are present against the run already these are populated already on the individual activity pages
- The user can then UPDATE the notes by pressing the save notes button
- When the user clicks upload photo the camera overlays the screen allowing the user to take a photo
- A URI file path to the image stored in AsyncStorage and these URI are rendered to the screen



Exercise Statistics

- There are a number of functions and calculations that are performed on the timing and location data to give a full set of activity statistics
- When the startRecording function is triggered, previous route data is reset and the current time/date at that moment is captured
- Whilst the activity is in progress a useEffect hook allows the the live timing data to be displayed
- When the users stops the recording it captures the end time and from this works out distance (as km/h) using the haversine formula (external plugin used) and speed
- These stats are then saved to AsyncStorage for use in the run details screen



```
const startRecording = async () => {  
  // reset route and start time  
  setRoute([]);  
  // get start time and save to state  
  setStartTime(new Date());  
}
```



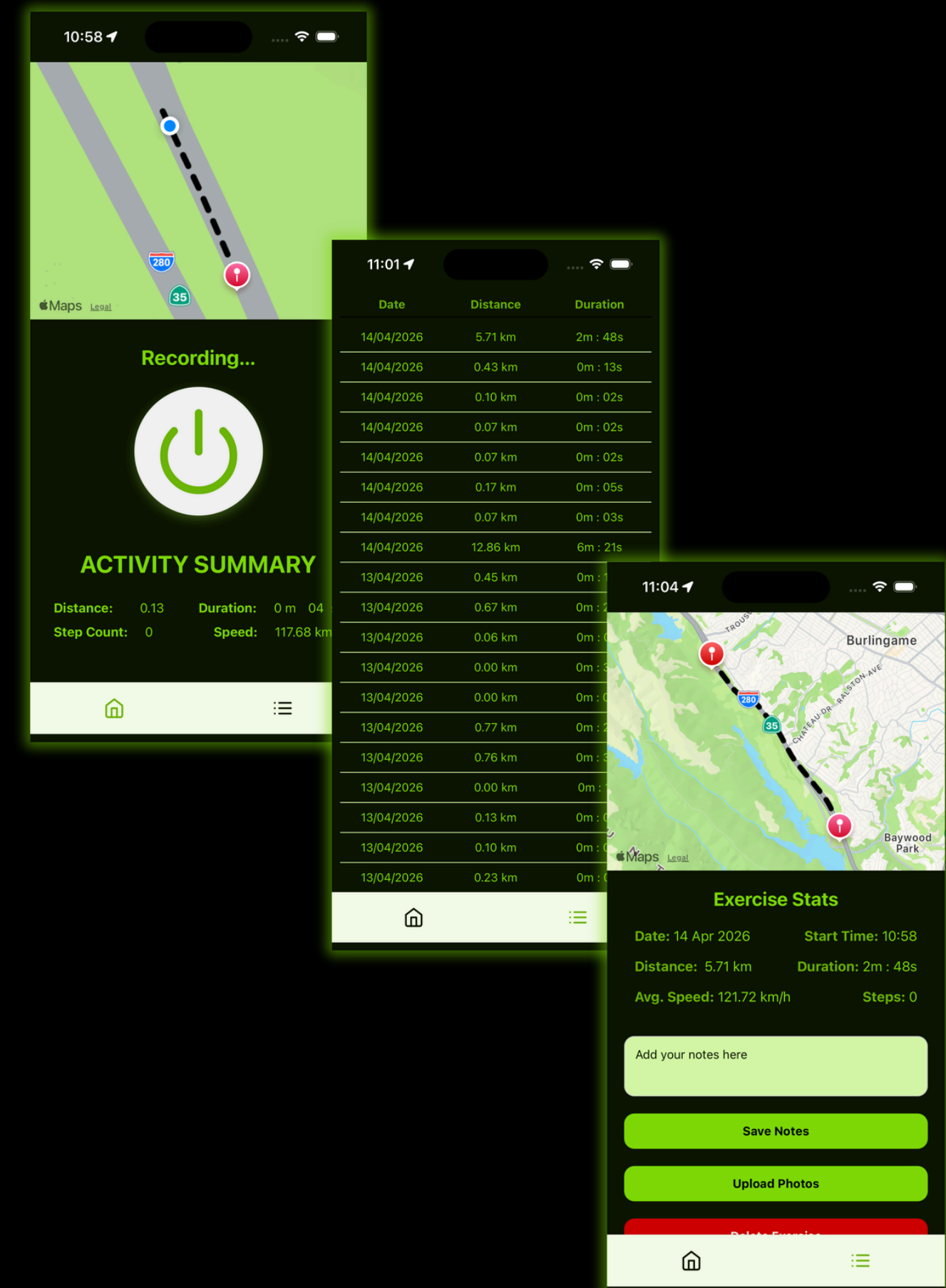
```
// timer showing duration while recording run  
useEffect(() => {  
  let interval;  
  
  if (recording && startTime) {  
    interval = setInterval(() => {  
      const now = new Date();  
      setDuration((now - startTime) / 1000);  
    }, 1000);  
  }  
  return () => clearInterval(interval);  
}, [recording, startTime]);
```



```
import haversine from "haversine-distance";  
  
export const calculateDistance = (coords) => {  
  if (!coords || coords.length < 2) return 0;  
  
  let total = 0;  
  for (let i = 1; i < coords.length; i++) {  
    total += haversine(coords[i - 1], coords[i]);  
  }  
  return total;  
};
```

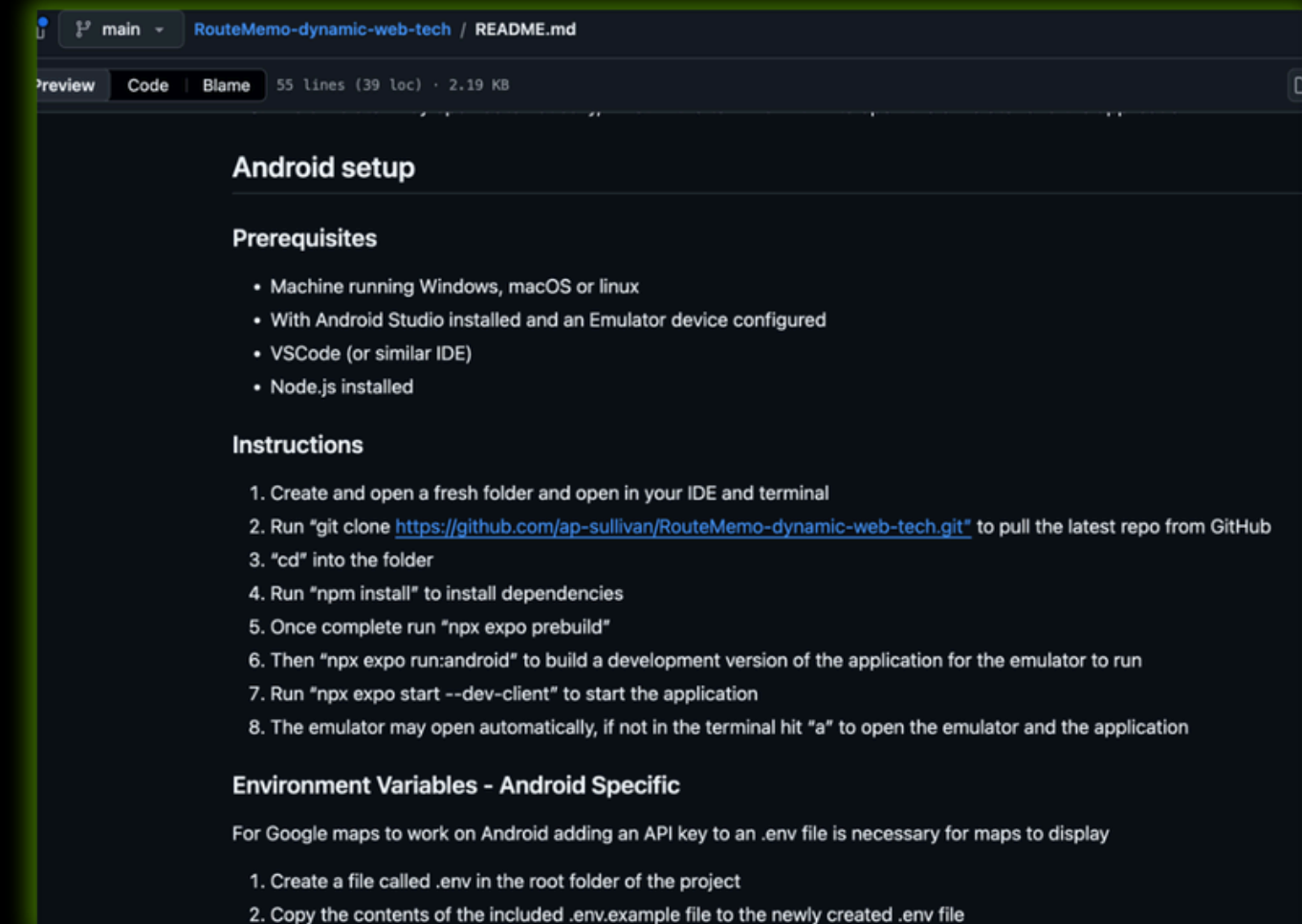

How to Use

- RouteMemo is very intuitive to use. After the application loads the user lands on the Start Screen. When the user starts their activity they press the start button to start recording their location and (on real devices) step data
- Live activity data is shown while the activity is recording and can be left on in the background while the activity is ongoing, the user can also still view and edit old activities at this time
- Once finished the user stops the recording to save the activity
- To see past activities the user can press the list icon in the bottom navigation to see a list of all the previously stored runs
- If the user then presses an individual activity run it will take them to the details screen where the stats for that run can be viewed, notes added or updated, and photos attached. The activity can also be dropped if not needed



Local Installation

- More detailed instructions, and the prerequisites needed to install RouteMemo are found in the Git repository README file
- To install RouteMemo locally follow the below command line instructions
- Creating a new folder and open in an IDE (VS Code recommended)
- Clone the GitHub repo, save, and move into the newly added project
- Install dependencies (“**npm install**”)
- Run a prebuild to generate native project files (“**npx expo prebuild**”)
- Build the app (“**npx expo run:ios**” OR “**npx expo run:android**”)
- Start the dev server so the app starts (“**npx expo start --dev-client**”)
- If installing the Android version, further steps need to be taken to ensure the Google Maps API key is added as an environment variable. Add this value into the .env file:
 - AlzaSyDEbDJQmoS6Y7dGj9SJ9Au7DIWLsRFt4aI



```
# This is an example of a .env file. Copy this to a .env file
GOOGLE_MAPS_API_KEY='add_google_maps_api_key_here'
```

Conclusions & Recommendations

- Using React Native for this type of application was a perfect fit as it meant I was able in effect to create two apps - one for iOS and one for Android using the exact same codebase
- I tried to keep the code as modular as possible but a refactor could definitely separate more logics especially for the AsyncStorage CRUD operations so they could be re-used and not just placed within the screens
- Understanding the separation of logics can be confusing at times but I think this application stays pretty true to the React best practices
- If I had more time I would have used AsyncStorage and syncing with an external database like Firebase Firestore making the data more secure from accidentally being wiped from the device
- This would also introduce dual use - guest mode for AsyncStorage and authenticated/ registered users being able to save to Firestore
- The design of the app is a modern futuristic look and doesn't need a dark mode as the theme is dark, however possibly adding a light mode for users could be implemented in future
- Another function I would like to implement is the ability to add photos from the device's album as presently it only accesses camera